

Retrieval-Augmented Generation (RAG) Overview

Introduction

Retrieval-Augmented Generation (RAG) is a technique that combines information retrieval with large language models (LLMs). Instead of relying only on the model's training data, RAG retrieves relevant information from external sources such as PDFs, websites, databases, and knowledge bases, then provides that information to the LLM before it generates an answer. This improves factual accuracy and allows AI systems to answer questions using private or up-to-date information.

How RAG Works

A typical RAG pipeline begins by loading documents using document loaders. The documents are split into smaller chunks using a text splitter. Each chunk is converted into a vector representation called an embedding and stored in a vector database. When a user asks a question, the query is also converted into an embedding. A retriever searches the vector database for the most similar chunks. These retrieved chunks are added to the prompt sent to the LLM, which then generates a context-aware response.

Main Components

1. Document Loaders – Read data from PDFs, Word files, websites, CSVs, or databases.
2. Text Splitters – Divide long documents into manageable chunks.
3. Embedding Models – Convert text into vectors for similarity search.
4. Vector Databases – Store and search embeddings efficiently.
5. Retriever – Finds the most relevant document chunks.
6. Large Language Model – Generates the final response using retrieved context.

Advantages

RAG reduces hallucinations, supports private knowledge bases, enables easy knowledge updates without retraining the model, and improves answer quality. It is widely used in enterprise chatbots, customer support, legal research, healthcare, education, and internal knowledge management systems.

Applications

Organizations use RAG for document question answering, policy assistants, financial reporting, technical documentation search, research assistants, and AI-powered customer service. Popular tools include LangChain, LlamaIndex, FAISS, ChromaDB, Pinecone, Cohere Embeddings, OpenAI Embeddings, and Hugging Face models.

Conclusion

RAG has become one of the most important architectures in Generative AI because it enables LLMs to provide accurate, relevant, and up-to-date answers by combining retrieval with generation. It is a core technology for building reliable AI applications in production.

Introduction

Retrieval-Augmented Generation (RAG) is a technique that combines information retrieval with large language models (LLMs). Instead of relying only on the model's training data, RAG retrieves relevant information from external sources such as PDFs, websites, databases, and knowledge bases, then provides that information to the LLM before it generates an answer. This improves factual accuracy and allows AI systems to answer questions using private or up-to-date information.

How RAG Works

A typical RAG pipeline begins by loading documents using document loaders. The documents are split into smaller chunks using a text splitter. Each chunk is converted into a vector representation called an embedding and stored in a vector database. When a user asks a question, the query is also converted into an embedding. A retriever searches the vector database for the most similar chunks. These retrieved chunks are added to the prompt sent to the LLM, which then generates a context-aware response.

Main Components

1. Document Loaders – Read data from PDFs, Word files, websites, CSVs, or databases.
2. Text Splitters – Divide long documents into manageable chunks.
3. Embedding Models – Convert text into vectors for similarity search.
4. Vector Databases – Store and search embeddings efficiently.
5. Retriever – Finds the most relevant document chunks.
6. Large Language Model – Generates the final response using retrieved context.

Advantages

RAG reduces hallucinations, supports private knowledge bases, enables easy knowledge updates without retraining the model, and improves answer quality. It is widely used in enterprise chatbots, customer support, legal research, healthcare, education, and internal knowledge management systems.

Applications

Organizations use RAG for document question answering, policy assistants, financial reporting, technical documentation search, research assistants, and AI-powered customer service. Popular tools include LangChain, LlamaIndex, FAISS, ChromaDB, Pinecone, Cohere Embeddings, OpenAI Embeddings, and Hugging Face models.

Conclusion

RAG has become one of the most important architectures in Generative AI because it enables LLMs to provide accurate, relevant, and up-to-date answers by combining retrieval with generation. It is a core technology for building reliable AI applications in production.

Introduction

Retrieval-Augmented Generation (RAG) is a technique that combines information retrieval with large language models (LLMs). Instead of relying only on the model's training data, RAG retrieves relevant information from external sources such as PDFs, websites, databases, and knowledge bases, then provides that information to the LLM before it generates an answer. This improves factual accuracy and allows AI systems to answer questions using private or up-to-date information.

How RAG Works

A typical RAG pipeline begins by loading documents using document loaders. The documents are split into smaller chunks using a text splitter. Each chunk is converted into a vector representation called an embedding and stored in a vector database. When a user asks a question, the query is also converted into an embedding. A retriever searches the vector database for the most similar chunks. These retrieved chunks are added to the prompt sent to the LLM, which then generates a context-aware response.

Main Components

1. Document Loaders – Read data from PDFs, Word files, websites, CSVs, or databases.
2. Text Splitters – Divide long documents into manageable chunks.
3. Embedding Models – Convert text into vectors for similarity search.
4. Vector Databases – Store and search embeddings efficiently.
5. Retriever – Finds the most relevant document chunks.
6. Large Language Model – Generates the final response using retrieved context.

Advantages

RAG reduces hallucinations, supports private knowledge bases, enables easy knowledge updates without retraining the model, and improves answer quality. It is widely used in enterprise chatbots, customer support, legal research, healthcare, education, and internal knowledge management systems.

Applications

Organizations use RAG for document question answering, policy assistants, financial reporting,

technical documentation search, research assistants, and AI-powered customer service. Popular tools include LangChain, LlamaIndex, FAISS, ChromaDB, Pinecone, Cohere Embeddings, OpenAI Embeddings, and Hugging Face models.

Conclusion

RAG has become one of the most important architectures in Generative AI because it enables LLMs to provide accurate, relevant, and up-to-date answers by combining retrieval with generation. It is a core technology for building reliable AI applications in production.